Yes, and I actually recommend doing it this way for your thesis/project.

Instead of one huge pipeline diagram, create **one diagram for each notebook cell**. It makes the workflow much easier to understand, and reviewers can see exactly what each stage does.

I would structure it like this:

Cell 5 – XML Cleaning

Raw XML Documents

|

▼

Remove Duplicates

|

▼

Handle Null Values

|

▼

Clean Text

(Whitespace, CDATA, etc.)

|

▼

Clean XML Data

Cell 6 – Knowledge Node Creation

Clean XML Data

|

▼

One XML = One Knowledge Node

|

┌──────────┼──────────┐

▼ ▼ ▼

Node ID Title Raw Content

|

▼

Knowledge Nodes

Cell 7 – Hyperlink Extraction

Knowledge Nodes

|

▼

Read XML Content

|

▼

Find [[Hyperlinks]]

|

▼

Parse Target References

|

▼

Hyperlink Records

Cell 8 – Sentence Matching

Hyperlink Records

|

▼

Split XML into Sentences

|

▼

Find Sentence containing Link

|

▼

Source | Target | Sentence

Cell 9 – Semantic Relationship Extraction

Sentence Records

|

▼

Azure OpenAI LLM

|

▼

Extract Semantic Predicate

|

▼

Source – Predicate – Target

Example

Engine ----depends_on----> Fuel System

Cell 10 – RDF Triple Generation

Semantic Records

|

▼

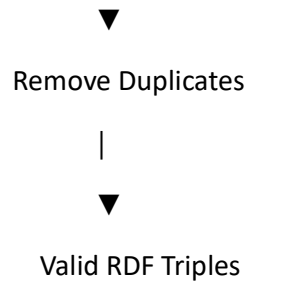
Remove Null Triples

|

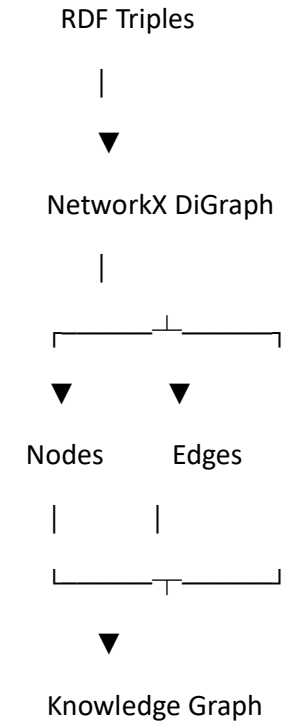
▼

Remove Self Loops

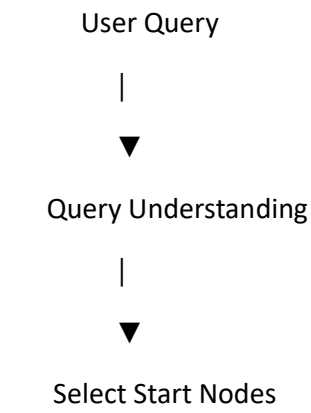
|

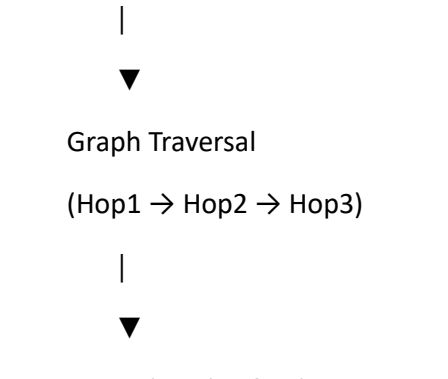


Cell 11 – Knowledge Graph Construction

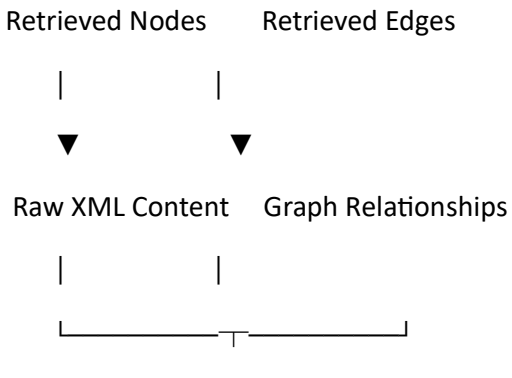


Cell 16 – Graph Retriever

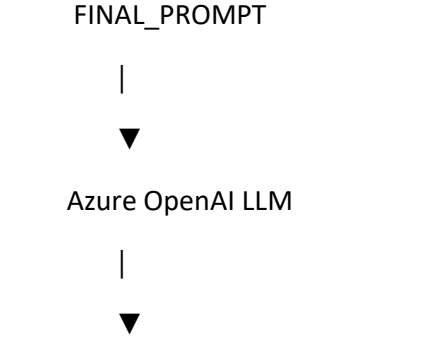


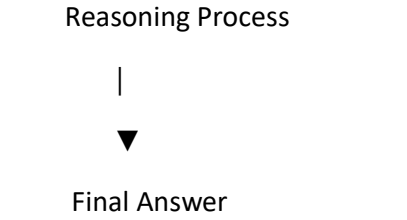


Cell 17 – Context Builder

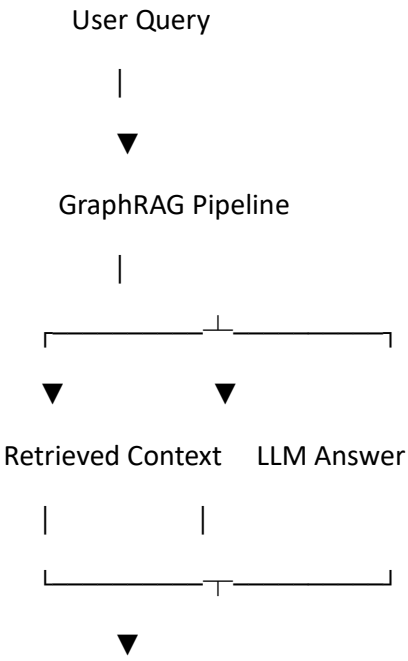


Cell 18 – LLM Generation





Cell 19 – MLflow Evaluation



Final Architecture Diagram

Finally, create one summary diagram that connects all the cells:

OFFLINE PHASE

Raw XML

|

▼

Cell 5 XML Cleaning

|

▼

Cell 6 Knowledge Nodes

|

▼

Cell 7 Hyperlink Extraction

|

▼

Cell 8 Sentence Matching

|

▼

Cell 9 Semantic Predicate Extraction

|

▼

Cell10 RDF Triple Generation

|

▼

Cell11 Knowledge Graph

ONLINE PHASE

User Query

|

▼

Cell16 Graph Retrieval

|

▼

Cell17 Context Builder

|

▼

Cell18 Azure OpenAI LLM

|

▼

Generated Answer

|

▼

Cell19 MLflow Evaluation

Recommendation

For your project report or thesis, I recommend **redrawing these as professional flowcharts** (using diagrams with boxes, arrows, colors, and icons) rather than keeping them as plain text. That would look much more polished and publication-quality.

resentation: Enterprise GraphRAG Proof of Concept

Title: Intelligent Retrieval for XML Knowledge Bases: A Hybrid GraphRAG Approach **Presenter:** [Your Name]

Slide 1: Executive Summary

Visual: A simple icon showing a Database, a Brain, and a Graph connected together. **Talking Points:**

- **The Challenge:** Scaling our documentation search across 20,000+ XML files accurately. Standard keyword search fails when users don't use exact terminology.
- **The Solution:** We built a Proof of Concept (PoC) for a **Hybrid GraphRAG** system.
- **The Result:** It perfectly blends semantic AI search with structural Knowledge Graphs to provide 100% accurate, hallucination-free answers.

Slide 2: The Limitation of Standard RAG & Keyword Search

Visual: A bulleted list showing pain points. **Talking Points:**

- Standard Keyword Search breaks down at scale (e.g., scoring a 0 when a user searches "make a test" but the document is named "Environment Setup").
- Standard RAG struggles with scattered context. If the answer requires reading 3 different connected documents, standard RAG fails to connect the dots.
- We needed a system that understands the *meaning* of words and the *relationships* between documents.

Slide 3: The "Two Brains" Architecture (The Hybrid Approach)

Visual: The POC_Master_Architecture.md Mermaid Diagram. **Talking Points:**

- To solve this, we architected a system with "Two Brains" that run in parallel:
- **1. The Vector Database (Speed & Semantic Brain):** Uses AI embeddings to instantly understand the "vibe" and meaning of a user's question, bypassing keyword limitations.
- **2. The Knowledge Graph (Logic & Structure Brain):** Uses explicit document hyperlinks to map exactly how every file connects to each other.

Slide 4: Phase 1 - Knowledge Graph Construction

Visual: A code snippet of PySpark cleaning XML, or a small 3-node graph. **Talking Points:**

- We engineered a 15-step PySpark pipeline in Databricks.

- **Data Engineering:** We dynamically parsed messy XML files, cleaning headers, macros, and hidden wiki syntax.
- **Predicate Extraction:** We leveraged Azure OpenAI to read the exact sentences containing hyperlinks and assign mathematical relationships (e.g., owned_by, uses).
- **Graphing:** We compiled these into RDF Triples and built a high-speed, in-memory NetworkX graph.

Slide 5: Phase 2 - Hybrid Online Retrieval

Visual: The Hybrid_GraphRAG_Architecture.md workflow diagram (Ask Vector -> Pass Top 5 -> Hop Graph). **Talking Points:**

- When a user asks a question, the system executes a 3-step retrieval:
- **Step 1:** The Vector Database receives the question and instantly finds the Top 5 most semantically relevant documents.
- **Step 2:** It passes those 5 IDs to the Knowledge Graph as "Start Nodes".
- **Step 3:** The Graph executes a 1-Hop and 2-Hop traversal along the arrows, gathering all deeply connected context and sentences.

Slide 6: Phase 3 - Databricks MLflow Evaluation

Visual: A screenshot of a Databricks MLflow table showing 1.0 scores. **Talking Points:**

- We didn't just guess if it worked; we mathematically proved it.
- We packaged the pipeline into a Databricks LLM-as-a-Judge evaluation framework.
- On our initial batch of 50 complex XML files, the strict MLflow Judge awarded us perfect 1.0 scores for both chunk_relevance and relevance_to_query.

Slide 7: Strategic Business Advantages

Visual: A comparison table (Our GraphRAG vs Microsoft GraphRAG). **Talking Points:**

- **Massive Cost Savings:** Unlike standard "Microsoft GraphRAG" which pays an LLM to read every word to guess entities, we used fast Python regex to extract *explicit* Wiki hyperlinks. We only used the LLM for lightweight relationship tagging.

- **Zero Hallucination Structure:** Because we built the graph on actual human-authored hyperlinks, the structural map is 100% accurate. The AI cannot invent fake connections.

Slide 8: Next Steps & Path to Production

Visual: An arrow pointing upwards (Scaling). **Talking Points:**

- **Finalize Vector Integration:** Complete the codebase link between the colleague's Databricks Vector Store and the Graph Retrieval engine.
- **Scale the Graph Engine:** For this PoC, we used NetworkX (in-memory). For full 20,000+ file production, we recommend migrating the graph to an enterprise **Neo4j** database server.
- **Deploy:** Deploy the final PySpark pipeline as a scheduled Databricks job.